

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO5 ASSESSMENT TOOL 1 – RESEARCH PAPER REVIEW / LITERATURE SURVEY

Course Code:	CSA10	Course Title:	Software Engineering
CO Assessed:	CO5	Assessment:	Assessment Tool 1 – Research Paper Review / Literature Survey
Weightage:	20%	Total Marks:	25
CO5 Statement:	Evaluate emerging technologies and societal impacts, maintaining and evolving software systems responsibly		
Student Name:	RAGUL SG	Reg. No.:	192525007
Date:		Duration:	60 minutes

Code of Conduct

I RAGUL SG – 192525007 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.



Signature of the Student

ABSTRACT

Software systems are increasingly delivered through automated, cloud-based pipelines while simultaneously becoming harder to maintain because of legacy constraints, architectural complexity, technical debt, data-intensive workloads, and the rapid adoption of AI. This literature survey examines how Sustainable DevOps and Green IT practices can be integrated with software maintenance and evolution. Eight scholarly and authoritative sources were screened, with six primary studies selected for detailed comparison and two foundational sources used to frame measurement and green software engineering. The review finds that energy efficiency is no longer only an infrastructure concern: it is influenced by algorithms, code structure, build configuration, test execution, deployment location, hardware lifetime, and functional workload. The strongest practical evidence concerns closed-loop energy measurement in CI/CD, dependency caching, software carbon-intensity measurement, carbon-aware scheduling, and structured cloud-sustainability governance. However, existing research remains fragmented across languages, tools, cloud providers, and organizational contexts. The principal research gap is the absence of a validated multi-objective control loop that jointly manages carbon intensity, technical debt, reliability, security, cost, and delivery speed. The report proposes a research agenda centered on standardized telemetry, carbon-aware quality gates, sustainability-aware refactoring, cross-cloud benchmarking, and longitudinal industrial evaluation.

Keywords: Sustainable DevOps, Green IT, software maintenance, technical debt, CI/CD, software carbon intensity, cloud sustainability, carbon-aware computing.

1. INTRODUCTION

Software maintenance and evolution traditionally emphasize corrective, adaptive, perfective, and preventive change. In modern cloud-native environments, these activities occur continuously through version control, automated testing, infrastructure as code, container orchestration, and observability. The same automation that improves release frequency can also multiply computation: every commit may trigger builds, tests, artifact generation, security scans, container images, and deployments. Consequently, a sustainable software engineering perspective must examine not only whether a pipeline is fast and reliable, but also how much energy and carbon it consumes per meaningful unit of service.

This report selects the topic “Carbon-Aware DevOps for Sustainable Software Evolution” because it directly connects the Unit 5 themes. Technical debt and legacy systems affect the energy and operational efficiency of software; emerging technologies such as AI, edge computing, blockchain, and cloud platforms change the resource profile of applications; and Sustainable DevOps provides the delivery mechanism through which measurement and improvement can become routine. The review is guided by the question: How can software teams integrate energy and carbon measurement into CI/CD and legacy-system evolution without sacrificing delivery speed, reliability, maintainability, security, or fairness?

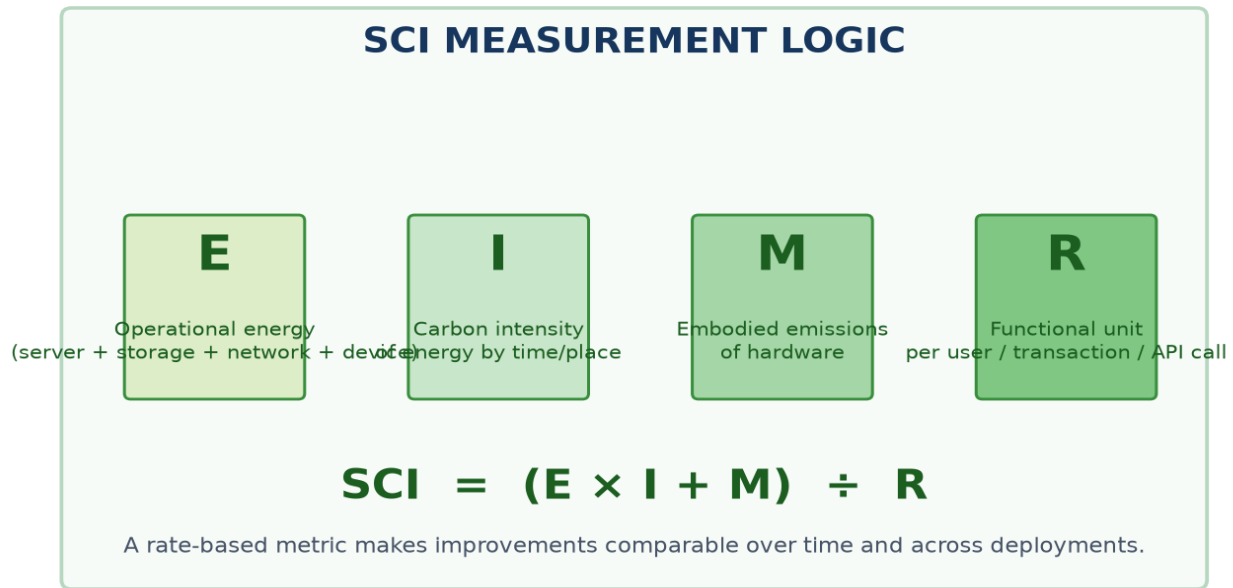


Figure 2. Software Carbon Intensity (SCI) measurement logic, adapted from the Green Software Foundation specification [8].

2. REVIEW SCOPE AND SELECTION APPROACH

The review used a focused narrative-survey approach. Sources were selected for direct relevance to at least one of four dimensions: software energy efficiency, sustainable CI/CD, sustainable cloud computing, or technical-debt management during software evolution. Preference was given to peer-reviewed journal and conference papers, supplemented by the ISO-accredited Software Carbon Intensity specification because it is an important operational standard rather than a conventional research paper. The final evidence set includes systematic literature reviews, empirical studies, a multivocal literature review, a scheduling study, and an authoritative measurement framework.

The analysis extracted each source’s research objective, methodology, technology or analytical framework, dataset or evidence base, major findings, limitations, and unresolved research opportunities. The comparison is interpretive rather than a meta-analysis because the studies use different units, technologies, workloads, and outcome measures. For example, a CI-energy benchmark cannot be compared numerically with an organizational cloud-mitigation model without imposing assumptions that the original studies do not support.

3. CONCEPTUAL BACKGROUND

3.1 SOFTWARE MAINTENANCE, EVOLUTION, AND TECHNICAL DEBT

Technical debt is the future cost created by expedient or suboptimal design and implementation decisions. In legacy systems, debt may appear as obsolete dependencies, tightly coupled modules, undocumented business rules, duplicated code, fragile tests, unsupported platforms, or architecture

that resists incremental change. The maintenance challenge is not simply to rewrite the system: replacement may be too risky, expensive, or disruptive. Incremental refactoring, strangler-style modernization, characterization testing, interface stabilization, modularization, and observability are therefore important ways to improve the system while keeping it operational.

The recent multivocal review by Moreschini and colleagues shows that AI-enabled systems introduce additional forms of debt, including data, infrastructure, pipeline, explainability, prompt, and governance debt [4]. This finding is relevant to Green DevOps because inefficient retraining, oversized models, repeated data processing, and opaque operational decisions can increase both maintenance burden and resource consumption. Sustainability should therefore be treated as a quality attribute and a prioritization signal for refactoring, not as an isolated environmental activity.

3.2 GREEN SOFTWARE AND SUSTAINABLE DEVOPS

Green software engineering considers the environmental effects of software across its lifecycle. The field links software design, runtime behavior, hardware use, data movement, energy sources, and end-of-life considerations [7]. Sustainable DevOps extends this lifecycle view into the delivery system. It asks teams to make sustainability visible in backlog planning, code review, build configuration, test selection, deployment policies, infrastructure provisioning, monitoring, and incident learning.

The SCI specification operationalizes this idea by expressing carbon intensity as a rate: $SCI = (E \times I + M) \div R$, where E is operational energy, I is the carbon intensity of electricity, M is amortized embodied emissions, and R is a functional unit such as a transaction, user, or API call [8]. A rate is useful because total emissions alone may rise simply because a service grows. A lower rate can indicate a genuine efficiency improvement even while demand expands, although the choice of functional unit and allocation method remains consequential.

3.3 EMERGING TECHNOLOGIES AND SUSTAINABILITY

AI and machine learning can improve defect prediction, test prioritization, anomaly detection, and refactoring assistance, but they also create data and model lifecycle costs. Blockchain introduces replicated computation and storage, so the sustainability profile depends strongly on the consensus mechanism and workload. Edge computing can reduce network traffic and latency by processing data near its source, but it can increase the number of distributed devices that must be powered, updated, secured, and eventually replaced. Quantum computing is still exploratory for most software-engineering applications; its long-term sustainability cannot be inferred from theoretical speedups alone because hardware cooling, access infrastructure, algorithmic suitability, and total workflow energy must be evaluated.

These technologies reinforce a central lesson: sustainability is socio-technical. The environmental result depends on architecture, algorithms, operations, governance, user demand, hardware procurement, and ethical choices. Energy optimization that reduces accessibility, worsens reliability, or shifts impact to e-waste may not be sustainable in a broader sense.

4. INDIVIDUAL PAPER REVIEWS

4.1 KRUGLOV, SUCCI, AND VASUEZ (2021)

Kruglov, Succi, and Vasuez propose a method for incorporating energy-efficiency measurement into the CI/CD pipeline [1]. Their central objective is to connect development-stage software analysis with the efficiency of the product during operation. The approach combines static analysis and software metrics with power-consumption analysis, producing a closed-loop process that can identify code and runtime bottlenecks and recommend improvements. The contribution is important because it moves energy from an after-the-fact infrastructure report into a development feedback mechanism. Its principal limitation is practical generalizability: reliable measurements depend on instrumentation, repeatable workloads, runner hardware, and language/tool support. The study also leaves open how energy results should be balanced against build time, test coverage, reliability, and developer experience.

4.2 ALWAGEED ET AL. (2024)

Alwageed and colleagues investigate sustainable cloud-computing challenges using a systematic review, questionnaire survey, interpretive structural modeling, artificial neural networks, and a real-world case evaluation [2]. The study identifies 11 challenges and 66 practices, organizing mitigation around requirements, quality of service and service-level agreements, complexity and cybersecurity, and trust. Its strength is its explicit recognition that cloud sustainability is not reducible to electricity consumption: adoption barriers, security, quality, organizational capability, and trust influence whether sustainable practices survive operational reality. The main limitation is that the model is built substantially from expert and organizational evidence rather than continuous telemetry from diverse production systems. It therefore provides a valuable governance map but not a standardized engineering benchmark.

4.3 WYSOCKI, MICIULA, AND PLECKA (2025)

Wysocki, Miciuła, and Plecka provide a systematic review of methods for improving software energy efficiency and discuss the state of practice [3]. Their review emphasizes that energy use is shaped by software algorithms, code structure, runtime behavior, hardware selection, and measurement. This broad lifecycle perspective is useful for maintenance because refactoring can affect both maintainability and resource demand. The study's limitation is breadth: it surveys software energy efficiency broadly and provides less direct evidence about how CI/CD governance, developer incentives, cloud-region choice, or release policies can institutionalize these techniques. It therefore supports the need for a delivery-oriented research agenda.

4.4 MORESCHINI ET AL. (2026)

Moreschini et al. conduct a multivocal literature review of 61 primary sources on the evolution of technical debt from DevOps to generative AI [4]. The review reports that data, infrastructure, and pipeline debt are especially prevalent in machine-learning systems, and that MLOps practices are increasingly necessary for managing changing data dependencies and model retraining. It also

highlights emerging explainability, prompt, and governance debt. The paper is highly relevant to software evolution, but its focus is technical-debt management rather than carbon measurement. The unresolved issue is how to prioritize debt repayment when sustainability, reliability, security, maintainability, delivery speed, and cost pull in different directions.

4.5 ARNTZENIUS ET AL. (2026)

Arntzenius et al. establish a baseline for the energy consumption of continuous integration through repeated measurements of 204 open-source Java projects using Maven and Gradle [5]. Their results show a highly skewed distribution: most projects consume energy modestly, while a smaller CI-intensive group can accumulate annual energy footprints of hundreds of kilowatt-hours. The study also reports actionable savings from dependency caching, averaging about 30% in some Maven projects and exceeding 90% in some Gradle cases. The results make a strong case for pipeline-level optimization. However, the evidence is concentrated in Java, open-source repositories, and particular build systems; generalization to other languages, runners, test suites, organizations, and cloud regions requires further study.

4.6 BASHIR ET AL. (2024)

Bashir et al. examine carbon-aware job scheduling and question how carbon-footprint metrics treat embodied emissions [6]. Their analysis shows that scheduling decisions can be distorted by the treatment of “sunk” embodied carbon and that operational and embodied emissions should not be collapsed into an oversimplified decision rule. This is a valuable corrective to purely location-based or instantaneous carbon optimization. The limitation is that the work is principally about scheduling and placement; it does not fully specify how developer-facing CI/CD policies, release urgency, test quality, and service-level objectives should be integrated into carbon-aware decisions.

5. COMPARATIVE LITERATURE ANALYSIS

The reviewed studies differ in scale and evidence type. The systematic reviews establish the breadth of the problem, the empirical CI study demonstrates measurable pipeline interventions, the cloud study contributes an organizational mitigation model, the scheduling study improves carbon-accounting reasoning, and the technical-debt review explains why AI-era systems require new maintenance practices. Together they support a layered interpretation: measurement is necessary, optimization is possible, governance determines adoption, and software evolution determines whether gains persist.

AUTHOR / YEAR	RESEARCH PROBLEM	METHODOLOGY	DATASET / TOOLS	KEY FINDING	LIMITATION	RESEARCH GAP
Kruglov et al. (2021) [1]	Energy-aware CI/CD	Static analysis + power analysis; closed loop	Application/code metrics and energy measurements	Links development feedback to runtime energy bottlenecks	Instrumentation and cross-platform validation remain limited	Universal energy telemetry and multi-objective quality gates
Alwageed et al. (2024) [2]	Sustainable cloud adoption	SLR + survey + ISM + ANN + case	11 challenges, 66 practices, expert evidence	Maps organizational and technical mitigation	Survey/model evidence; limited direct telemetry	Longitudinal field validation with carbon data

				dependencies		
Wysocki et al. (2025) [3]	Software energy efficiency	Systematic literature review	Methods across software and hardware contexts	Shows energy is affected by design, algorithms, runtime and hardware	Broad coverage; less CI/CD governance evidence	Operationalize methods in delivery workflows
Moreschini et al. (2026) [4]	Technical debt in AI-era evolution	Multivocal review of 61 sources	Peer-reviewed + industry literature	Identifies data, infrastructure, pipeline, explainability and prompt debt	No unified carbon/debt optimization model	Sustainability-aware AI/MLOps debt management
Arntzenius et al. (2026) [5]	CI energy baseline	Repeated benchmark study	204 open-source Java projects; Maven/Gradle	Caching can produce substantial practical savings	Language/tool/ecosystem specificity	Cross-language, cross-runner benchmark suite
Bashir et al. (2024) [6]	Carbon-aware scheduling metrics	Scheduling and carbon-accounting analysis	Server/job placement scenarios	Clarifies operational vs embodied carbon trade-offs	Limited developer workflow integration	CI/CD policies that use robust carbon accounting

Table 1. Comparative review of the six primary studies.

WHERE SUSTAINABILITY ENTERS THE DELIVERY PIPELINE

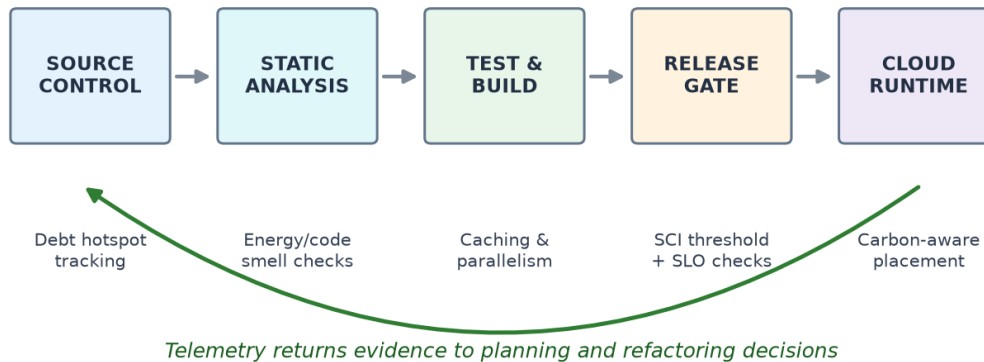


Figure 3. Points at which energy and carbon information can influence CI/CD decisions.

5.1 COMPARISON BY EVIDENCE AND PRACTICAL VALUE

The strongest direct operational evidence is currently found in measurement and optimization studies. CI energy baselines demonstrate that small configuration changes such as dependency caching can materially reduce repeated work [5]. The closed-loop CI/CD method shows how static and dynamic evidence can be combined [1]. SCI provides a common language for comparing operational and

embodied emissions per functional unit [8]. By contrast, cloud-sustainability and technical-debt studies contribute broader organizational and lifecycle explanations but require deeper integration with measured pipeline data [2] [4].

A second comparison concerns scalability. A local energy measurement can be precise for a single build but difficult to compare across hardware. An organizational model can scale conceptually but may be less sensitive to actual workload variation. A standardized rate such as SCI supports comparison, yet the result depends on the quality of telemetry, carbon-intensity data, embodied-emission assumptions, and functional-unit definition. A mature Green DevOps practice should therefore use multiple levels of evidence rather than treating one metric as sufficient.

6. LIMITATIONS AND CHALLENGES IN EXISTING APPROACHES

First, measurement remains inconsistent. Studies may report energy, power, runtime, carbon emissions, or estimated cost, and these quantities are not interchangeable. Hardware counters, cloud-provider estimates, container-level attribution, and model-based estimates have different accuracy and coverage. Repeatability is also difficult because CI runners, caches, background processes, workloads, and electricity carbon intensity change over time.

Second, sustainability objectives can conflict. A carbon-aware deployment may be slower, a highly replicated architecture may improve availability but increase embodied and operational impact, and a larger test suite may increase energy while reducing defect risk. Optimization must therefore be constrained by reliability, security, accessibility, performance, and service-level objectives. The literature rarely evaluates all of these outcomes together.

Third, technical debt is not yet routinely connected to environmental metrics. Teams may know that a dependency is obsolete or that a module is difficult to change, but they often cannot quantify whether refactoring it will reduce energy, build duration, data movement, or hardware demand. Conversely, an energy optimization may increase complexity and create new maintenance debt. This bidirectional relationship is an important unresolved challenge.

Fourth, emerging AI systems intensify lifecycle uncertainty. Model retraining, data drift, prompt changes, evaluation, explainability, and governance create recurring work that resembles technical debt. AI assistance may reduce individual coding effort but also increase code volume, dependency surface, review burden, and verification cost. Sustainable DevOps must evaluate the whole system rather than assuming that automation is automatically green.

7. IDENTIFIED RESEARCH GAP

The literature indicates a clear gap: there is no widely validated, multi-objective Green DevOps control loop that connects software maintenance decisions, technical-debt indicators, CI/CD energy, SCI-style carbon intensity, cloud placement, reliability, security, delivery performance, and organizational governance. Existing studies usually optimize one layer at a time. Pipeline research measures builds, cloud research models adoption, technical-debt research classifies maintenance

problems, and scheduling research examines carbon placement. Their insights are complementary, but the interfaces among them remain under-specified.

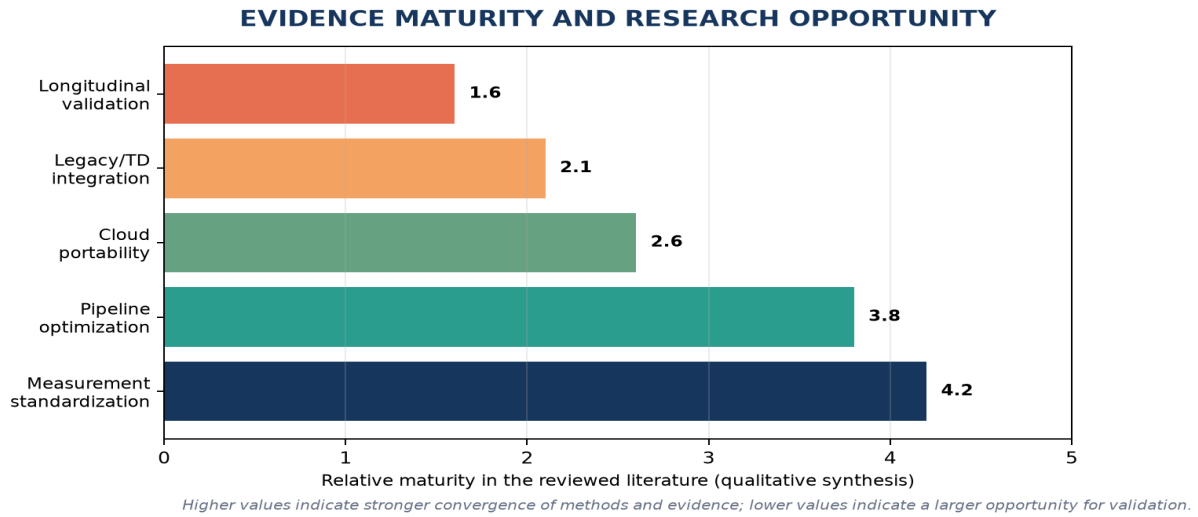


Figure 4. Qualitative synthesis of evidence maturity and opportunity for further validation.

A second gap concerns external validity. Evidence is often limited to one language, one build ecosystem, open-source repositories, laboratory workloads, expert surveys, or conceptual frameworks. A stronger evidence base would evaluate heterogeneous repositories and production-like workloads across multiple languages, CI runners, cloud regions, energy sources, and organizational sizes. It would also report trade-offs rather than only reductions, including build latency, failure rate, test coverage, developer effort, cost, and service quality.

8. PROPOSED IMPROVEMENTS AND FUTURE DIRECTIONS

DIRECTION	PROPOSED IMPROVEMENT	EXPECTED CONTRIBUTION
STANDARDIZED TELEMETRY	Adopt repeatable measurements for energy, runtime, workload, carbon intensity, hardware allocation and embodied-emission assumptions.	Comparable evidence across languages, runners and clouds.
CARBON-AWARE QUALITY GATES	Add SCI or energy thresholds to CI/CD alongside test, security, performance and reliability gates.	Sustainability becomes an actionable release criterion rather than a report.
SUSTAINABILITY-AWARE REFACTORING	Use technical-debt hotspots, energy profiles and dependency graphs to prioritize refactoring and modernization.	Debt repayment can target both maintainability and resource efficiency.
CARBON-AWARE SCHEDULING	Schedule deferrable builds, tests, training and batch jobs using time- and location-sensitive carbon signals while respecting deadlines.	Lower operational emissions without compromising urgent delivery.
CROSS-CLOUD BENCHMARKS	Compare managed services, containers, serverless, edge and legacy-hosting alternatives using consistent functional units.	Evidence-based architecture and modernization decisions.
LONGITUDINAL FIELD STUDIES	Measure changes over months, including reliability, cost, developer experience, technical debt and emissions.	Stronger causal evidence and realistic adoption guidance.

Table 2. Research and practice directions derived from the identified gaps.

A practical implementation sequence is to begin with baseline instrumentation, select a meaningful functional unit, and establish a dashboard that reports energy and carbon alongside delivery metrics. Teams can then optimize repeated work through dependency caching, incremental builds, test selection, artifact reuse, right-sizing, and workload scheduling. Only after measurement is stable should teams set thresholds, because premature targets can encourage gaming or shift emissions outside the measured boundary. Finally, sustainability data should become part of maintenance planning: modules with high change frequency, high energy demand, and high debt can receive priority for refactoring or architectural modernization.

9. CONCLUSION

This literature survey reviewed sustainable DevOps and Green IT through the connected lenses of software maintenance, evolution, emerging technologies, and cloud delivery. The literature demonstrates that software energy and carbon impact can be measured and improved, especially when teams use closed-loop analysis, dependency caching, efficient testing, carbon-aware scheduling, and a standardized rate such as SCI. At the same time, sustainability cannot be reduced to a single infrastructure setting. It is shaped by technical debt, architecture, algorithms, data, hardware lifetime, organizational incentives, security, reliability, and ethical responsibility.

The most important unresolved issue is integration. Future Green DevOps research should build and evaluate a multi-objective control loop in which carbon intensity informs, but does not override, software quality and societal requirements. Such a framework could help organizations evolve legacy systems responsibly, manage AI-era technical debt, and make cloud decisions that are measurable, explainable, and resilient. The result would be a more complete interpretation of software quality: software should be correct, secure, maintainable, reliable, affordable, and environmentally responsible.

10. REFERENCES

- [1] A. Kruglov, G. Succi, and X. Vasquez, "Incorporating energy efficiency measurement into CI/CD pipeline," in *Proceedings of the 2021 European Symposium on Software Engineering*, 2021, pp. 14–20. doi: 10.1145/3501774.3501777. Available: <https://doi.org/10.1145/3501774.3501777>
- [2] H. S. Alwageed, I. Keshta, R. A. Khan, A. Alzahrani, M. U. Tariq, and A. Ghani, "An empirical study for mitigating sustainable cloud computing challenges using ISM-ANN," *PLOS ONE*, vol. 19, no. 9, e0308971, 2024. doi: 10.1371/journal.pone.0308971. Available: <https://doi.org/10.1371/journal.pone.0308971>
- [3] W. Wysocki, I. Miciuła, and P. Plecka, "Methods of improving software energy efficiency: A systematic literature review and the current state of applied methods in practice," *Electronics*, vol. 14, no. 7, 1331, 2025. doi: 10.3390/electronics14071331. Available: <https://doi.org/10.3390/electronics14071331>
- [4] S. Moreschini et al., "The evolution of technical debt from DevOps to generative AI: A multivocal literature review," *Journal of Systems and Software*, vol. 231, 112599, 2026. doi: 10.1016/j.jss.2025.112599. Available: <https://doi.org/10.1016/j.jss.2025.112599>
- [5] R. Arntzenius et al., "On the energy consumption of continuous integration in open-source Java projects," *IEEE SANER 2026*. Available: <https://ieeexplore.ieee.org/document/11500151/>

- [6] N. Bashir et al., “The sunk carbon fallacy: Rethinking carbon footprint metrics for effective carbon-aware scheduling,” in Proceedings of the ACM Symposium on Cloud Computing, 2024. doi: 10.1145/3698038.3698542. Available: <https://doi.org/10.1145/3698038.3698542>
- [7] ACM, “Software engineering aspects of green and sustainable software engineering,” 2017. doi: 10.1145/3021460.3021464. Available: <https://doi.org/10.1145/3021460.3021464>
- [8] Green Software Foundation, “Software Carbon Intensity (SCI) Specification,” aligned with ISO/IEC 21031:2024. Available: <https://greensoftware.foundation/standards/sci/>
- [9] M. Fowler, Refactoring: Improving the Design of Existing Code, 2nd ed. Boston, MA, USA: Addison-Wesley, 2018.
- [10] D. Drescher, Blockchain Basics: A Non-Technical Introduction in 25 Steps. New York, NY, USA: Apress, 2017.
- [11] E. Rieffel and W. Polak, Quantum Computing: A Gentle Introduction. Cambridge, MA, USA: MIT Press, 2011.